TP3 Python

```
import numpy as np
```

EX1

```
#1
def pivot(A,b,j):
  l = len(A)
  index = j
  Max = abs(A[j][j])
  for i in range(j,l):
    if abs(A[i][j]) > Max :
      Max, index = abs(A[i][j]) , i    #Max = abs(A[i][j) and index = i
  if index != j :

    A[[index,j]]  =  A[[j,index]]
    b[[index,j]]  = b[[j,index]]#use numpy tables



#2
def elim_bas(A,b,j):
    l = len(A)
    pivot(A,b,j)
    for e in range(j+1,l):
      #pivot(A,b,j)
      factor = A[e][j]/A[j][j]

      A[e] = A[e]-A[j]*factor
      b[e] = b[e] - b[j] * factor



#3
def descente(A,b):
  c = len(A[0])
  for j in range(0,c):
    pivot(A,b,j)
    elim_bas(A,b,j)


Start coding or generate with AI.


A = np.array([[3,88,15],
              [10.0,5,20],
               [14,485,12]])
b = np.array([154.2,4,147])
c = A.copy()
d = b.copy()
remonte(A,b)

print(A)
```

```
     [[  -5.89834025    0.            0.        ]
      [ -13.33333333 -803.33333333    0.        ]
      [  14.          485.           12.        ]]
```

Ex2

```
#1
def elim_haut(A, b, j):
  l = len(A)
  pivot(A,b,j)
  for e in range(j-1, -1, -1):
    # pivot(A,b,j)
    factor = A[e][j]/A[j][j]
```

```
            A[e] = A[e]-A[j]*factor
            b[e] = b[e] - b[j] * factor
```

```
#2
def remonte(A,b):
  c = len(A[0])
  for j in range(c-1,-1,-1):
    elim_haut(A,b,j)
```

```
c
```

```
3
```

```
A = np.array([[3,88,15],
              [10,5,20],
               [14,485,12]])
b = np.array([152,4,147])
```

```
Start coding or generate with AI.
```

```
elim_haut(A,b,1)
A
```

```
array([[  0,   0,  12],
       [ 14, 485,  12],
       [ 10,   5,  20]])
```

```
remonte(A,b)
descente(a,b)
A
```

```
array([[ -4,   0,   0],
       [  8, 482,   0],
       [ 14, 485,  12]])
```

Ex3

```
#1
def solve_diagnol(A,b):
  l = len(A)
  x = np.zeros(l)
  for i in range(l):
    x[i] = b[i] / A[i][i]
  return x

#2
def gauss(A,b):
  l = len(A)
  A_copy = A.copy()
  b_copy = b.copy()
  descente(A_copy,b_copy)
  remonte(A_copy,b_copy)
  return  solve_diagnol(A_copy,b_copy)
```

```
A = np.array([[1,1],
              [3,-1]])
b = np.array([3,0])
```

```
gauss(A,b)
```

```
array([0.66666667, 2.        ])
```

```
c = np.dot(A,gauss(A,b))
c
#c 9ribe lil b donc fonction ti5dim
```

```
array([4., 0.])
```

Ex4

```
#1
def det(A):
  A_copy = A.copy()
  det_A = np.linalg.det(A_copy)
  return det_A
#2
def inverse(A):
  l = len(A)
  c = len(A[0])
  identity = np.identity(l)
  b = np.zeros(l)
  A_copy = A.copy()
  A_copy = np.hstack([A_copy, identity])
  for j in range(0,len(A_copy)):
    elim_bas(A_copy,b,j)
    elim_haut(A_copy,b,j)
  for i in range(l):
    A_copy[i] = A_copy[i]*1/A_copy[i][i]


  return A_copy[:,c:]
```

```
A = np.array([
    [1,2,3],
    [4,5,6],
    [7,8,1]
])
```

```
c = np.dot(inverse(A),A)
c
```

```
array([[ 1.00000000e+00,  2.22044605e-16,  4.44089210e-16],
       [ 0.00000000e+00,  1.00000000e+00, -2.22044605e-16],
       [ 1.11022302e-16,  0.00000000e+00,  1.00000000e+00]])
```

Start coding or generate with AI.

```
[[1 2 3]
 [7 8 1]
 [4 5 6]]
```

```
# exercice 5 mouch de5l
```

exercice 5 mouch de5l

Ex5

+ Code + Text

```
#1
def lower(A,b):
  l = len(A)
  c = len(A[0])

  x = [0]*l
  for i in range(l):
    x[i] = b[i]
    for j in range(i):
      x[i] = x[i] - A[i][j]* x[j]
    x[i] = x[i]/A[i][i]
  return x
#2
def jacobi(A,b,x0,eps,itmax):
  d = np.diagonal(A)
  e = 0
  for n in range(itmax):
    e +=1
    delta = np.dot(np.linalg.inv(d) , b - np.dot(A,x0))
    tol = np.linalg.norm(delta)

    if eps > abs(tol) :
```

```
      return print(f"iteration exit with succes with value % at iteration %",x0,n)#iteration exit with succes
      pass
    else:

      x0 = delta +x0
  if e == itmax:
    return print(f"iteration exit with faliure with value %",x0)
#3


def gauss_seidel(A, b, x0, eps, itmax):
    n = len(b)
    x = x0.copy()

    for _ in range(itmax):
        x_new = np.zeros_like(x)
        for i in range(n):
            x_new[i] = (1 / A[i, i]) * (b[i] - np.dot(A[i, :i], x_new[:i]) - np.dot(A[i, i+1:], x[i+1:]))

        if np.linalg.norm(x_new - x) < eps:
            return x_new

        x = x_new

    return x
```

mouch lezm 3lik tifhim el principe des fonctions, eme idhe t7ib t3rf tnjm tchouf fil youtube wele tas2lni privee